

Execute 4.0 Hackathon

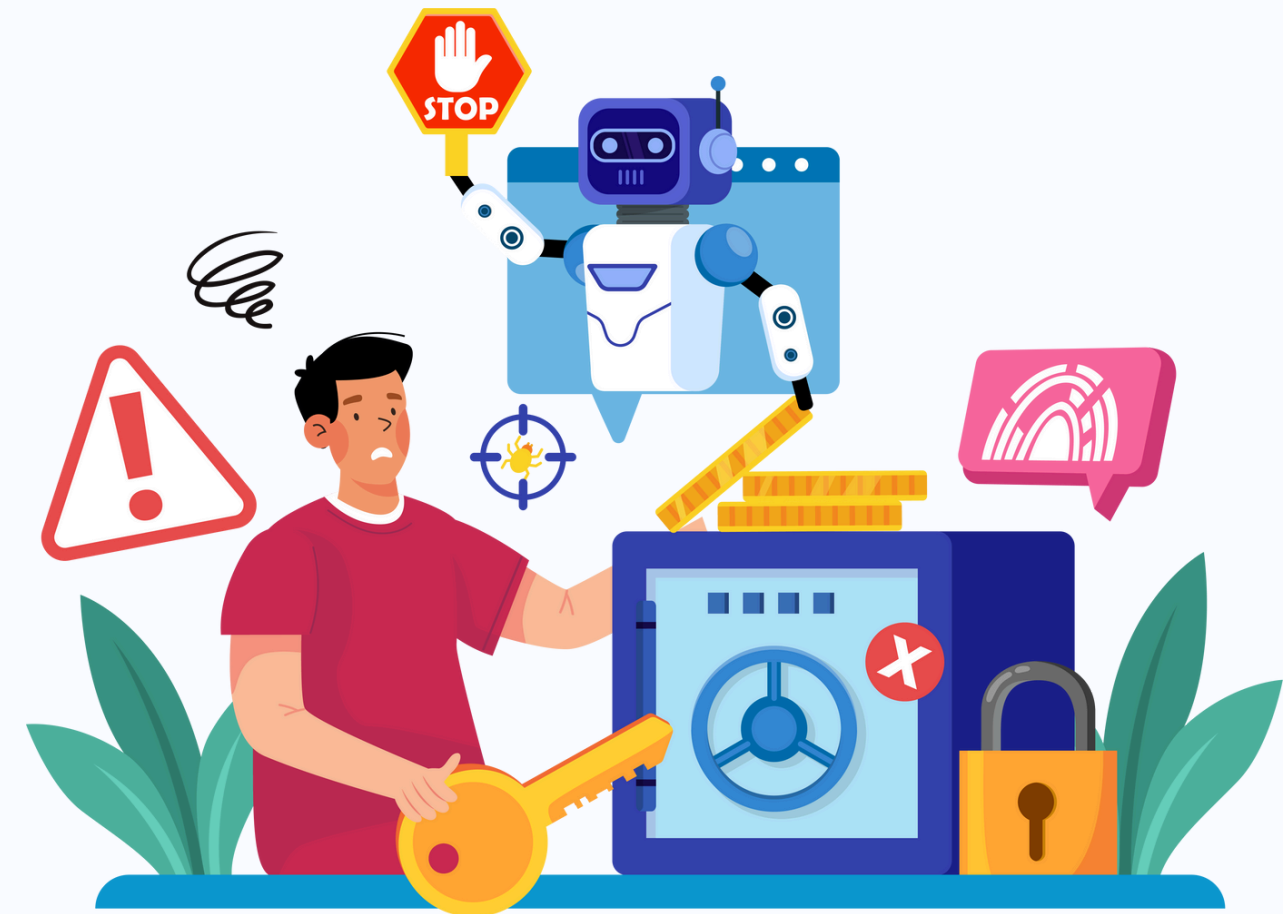
Fraud Detection, Alert, and Monitoring (FDAM) system

TEAM NAME - SPARK

Demonstration of Prototype Link

[Prototype Video Link](#)

Live Project Link - [Link](#)



What problem statement are we addressing

1 Fraudulent Transaction Detection: Identifying and preventing unauthorized or suspicious financial transactions using AI and machine learning.

2 Real-Time Fraud Prevention: Ensuring instant detection of fraudulent activities to minimize financial losses and protect users.

3 Anomaly Detection in Payments: Analyzing transaction patterns to flag unusual behavior, such as high-value transactions, rapid purchases, or device/location mismatches.

4 Handling Imbalanced Data: Developing a model that accurately detects fraud despite fraud cases being significantly fewer than legitimate transactions.

5 Building a Scalable Fraud Monitoring System: Creating a robust, scalable fraud detection system that integrates with payment gateways and financial institutions for continuous monitoring and security.



Our Solution

Core Solution -

1. Hybrid Fraud Detection – Combines rule-based logic, ML models (Random Forest, XGBoost, Neural Networks), and anomaly detection (Isolation Forest) for high-accuracy fraud detection.
2. Real-Time Detection – Response time under 300ms, enabling instant transaction verification for payment gateways.

Platform Components -

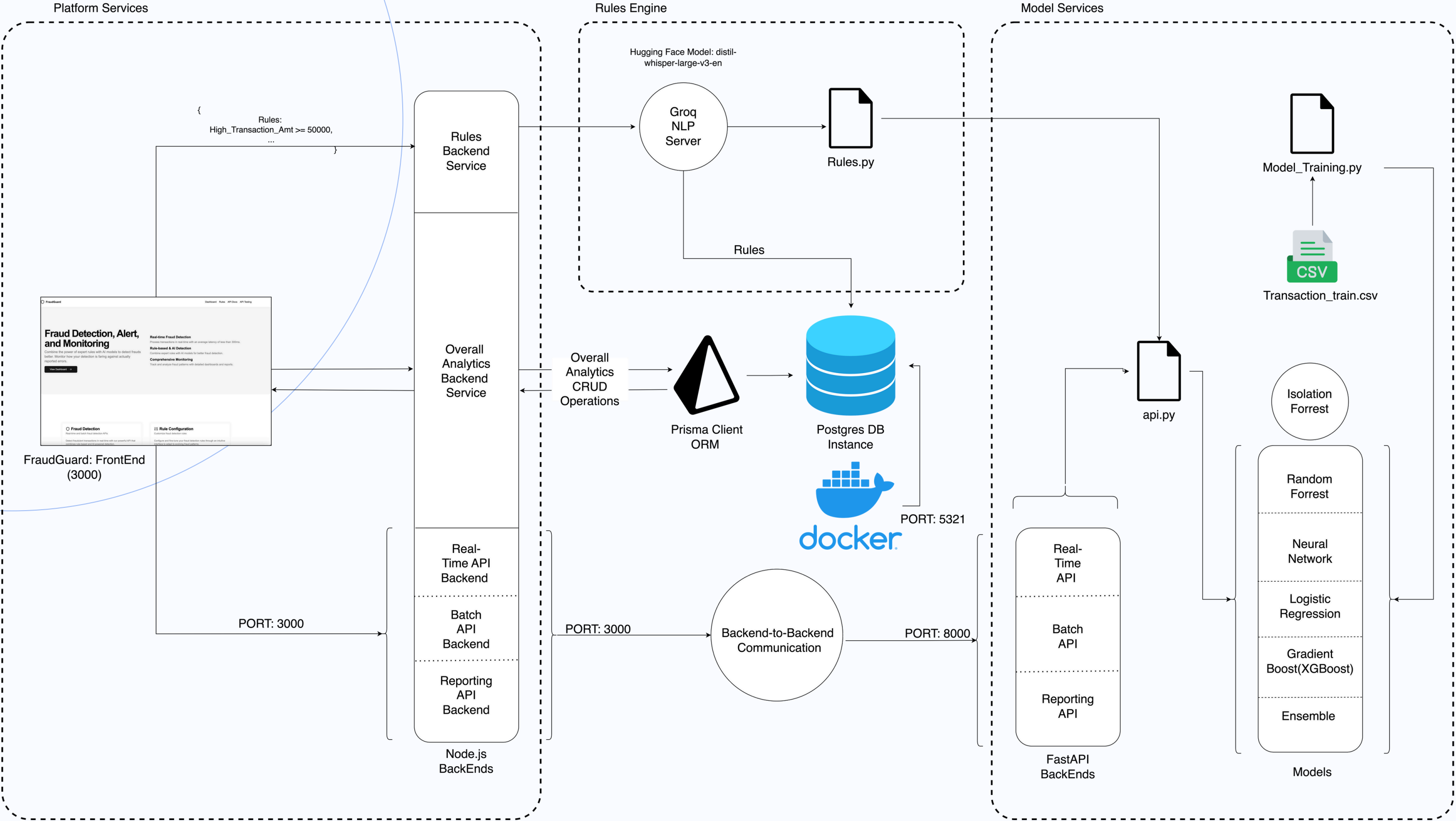
3. Comprehensive Monitoring & Insights – Interactive dashboard for fraud trends, transaction explorer for risk analysis, and rule configuration interface for customization.
4. Seamless API Integration – Well-documented APIs for real-time fraud detection, batch processing, and fraud reporting, with an interactive testing environment.

Key Differentiators -

5. Self-Improving & Explainable AI – Continuously learns from reported fraud cases while providing clear reasoning behind fraud assessments.
6. Scalable & Adaptive Architecture – Evolves with emerging fraud patterns, offers visualization tools, and ensures a user-friendly experience for fraud analysts.



Technical Architecture



Technologies / Tools Used

1. Frontend & UI

- Next.js with App Router, React, and TypeScript for robust UI development.
- Tailwind CSS and shadcn/ui for styling and responsive design.

2. Backend & APIs

- Next.js API routes with RESTful design for fraud detection.
- Server-side rendering for performance optimization.

3. Fraud Detection & Data Visualization

- Hybrid rule-based + AI detection with a configurable rule engine.
- Chart.js and custom visualizations for real-time fraud insights.

4. Testing & Monitoring

- Built-in API testing tools and model evaluation dashboard.
- Performance tracking with fraud detection accuracy metrics.

How we stand out from existing alternatives

Advanced Fraud Detection Mechanism

The platform combines rule-based detection with AI/ML models to enhance fraud identification accuracy. Rule-based detection allows for instant fraud flagging, while AI models conduct deeper analysis to detect evolving fraud patterns. A continuous feedback loop refines detection by learning from reported fraud cases, ensuring improved accuracy over time.

Intelligent Monitoring & Customization

Interactive dashboards provide real-time fraud trends and multi-dimensional transaction analysis for better monitoring. A no-code rule builder allows fraud analysts to create, test, and deploy rules without developer intervention. The integrated testing environment includes built-in API testing tools and a sample transaction generator, enabling instant evaluation of rule effectiveness.

Scalable & User-Centric Design

The system offers a performance evaluation suite with tools like a confusion matrix, feature importance visualization, and historical fraud comparisons to improve fraud detection accuracy. The intuitive interface ensures ease of use for fraud analysts. With a scalable API-first architecture, the platform supports real-time and batch processing, continuously improving detection through a fraud feedback loop.

Competitive Analysis

Standing Out in the Market

Traditional Rule-Based Systems (e.g., legacy fraud engines)

- Often rigid and static, relying on fixed thresholds and patterns.
- ✓ Our solution combines rules with ML + anomaly detection, allowing it to evolve and adapt to unseen fraud patterns.

Single Model ML Solutions

- Many fraud detection systems rely on just one model (e.g., only Random Forest or only Logistic Regression).
- ✓ We leverage a multi-model ensemble + anomaly detection for better recall on rare fraud cases while reducing false positives.

Anomaly Detection-Only Tools

- Pure anomaly detection solutions like Isolation Forests alone can have high false positive rates.
- ✓ By blending anomaly detection with supervised models and business rules, our system balances precision and recall more effectively.

Off-the-shelf SaaS Fraud Tools (e.g., Stripe Radar, Sift)

- Limited flexibility and expensive for businesses wanting custom fraud logic.
- ✓ Our approach is fully customizable, allowing teams to tweak rules, thresholds, and models for domain-specific fraud scenarios.

WireFrames and Platform

<https://execute-4-0.vercel.app>

API Documentation

Documentation for the Fraud Detection, Alert, and Monitoring APIs.

Real-time API

Batch API

Reporting API

Real-time Fraud Detection API

Process a single transaction and get fraud detection results in real-time.

Endpoint

POST /api/fraud/detect

Request Body

```
{
  "transaction_id": "txn_12345",
  "transaction_amount": 5000,
  "transaction_date": "2025-03-21T13:45:00Z",
  "transaction_channel": "web",
  "transaction_payment_mode": "Card",
  "payment_gateway_bank": "Example Bank",
  "payer_email": "user@example.com",
  "payer_mobile": "1234567890",
  "payer_card_brand": "Visa",
  "payer_device": "device_id_123",
  "payer_browser": "Chrome",
  "payee_id": "payee_123"
}
```

Response

```
{
  "transaction_id": "txn_12345",
  "is_fraud": false,
  "fraud_source": "rule",
  "fraud_reason": "",
  "fraud_score": 0.15
}
```

FraudGuard

Fraud Detection, Alert, and Monitoring

Combine the power of expert rules with AI models to detect frauds better. Monitor how your detection is faring against actually reported errors.

View Dashboard

Real-time Fraud Detection

Process transactions in real-time with an average

Rule-based & AI Detection

Combine expert rules with AI models for better

Comprehensive Monitoring

Track and analyze fraud patterns with detailed

Fraud Detection

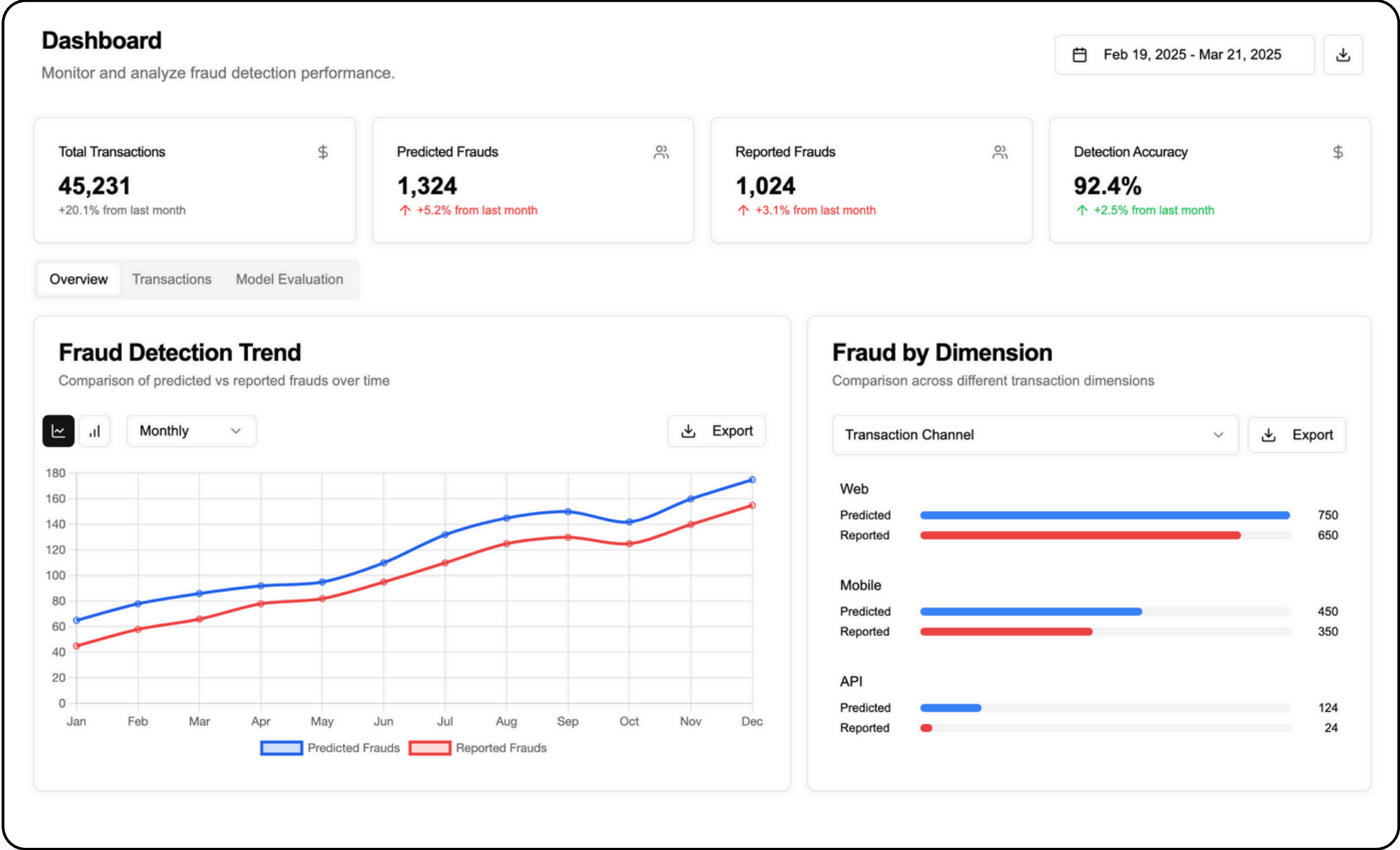
Real-time and batch fraud detection APIs

Rule Configuration

Customize fraud detection rules

WireFrames and Platform

<https://execute-4-0.vercel.app>



WireFrames and Platform

<https://execute-4-0.vercel.app>

Fraud Detection Rules

Configure and manage fraud detection rules and model settings.

Rule EngineModel SettingsRule Testing

Rule Engine Configuration

Configure rules to detect fraudulent transactions based on specific criteria.

Rule Execution Order

Rules are evaluated in the order they appear. Drag to reorder rules.

Transaction Amount Rules

Rules based on transaction amount and user spending patterns

High Amount Transaction

transaction_amount > 50000

Unusual Spending Pattern

transaction_amount > avg_user_transaction * 5

Add Rule

Location & Device Rules

Rules based on user location, IP address, and device information

Location Mismatch

user_country != transaction_country

New Device

is_new_device == true && transaction_amount > 10000

Bin

Ruleset Active

Save Changes

WireFrames and Platform

<https://execute-4-0.vercel.app>

Real-time APIBatch APIReporting API

Real-time Fraud Detection

Test the real-time fraud detection API by submitting a sample transaction.

Transaction ID

txn_2443

Amount

5000

Date

2025-03-21T22:39:19Z

Channel

Web

Payment Mode

Card

Bank

Example Bank

Email

user@example.com

Mobile

1234567890

Card Brand

Visa

Payee ID

payee_123

Device ID

device_id_123

Browser

Chrome

Test Fraud Detection →

API Request & Response

View the request and response data.

Request

Copy

```
{  "transaction_id": "txn_2443",  "transaction_amount": 5000,  "transaction_date": "2025-03-21T22:39:19Z",  "transaction_channel": "web",  "transaction_payment_mode": "Card",  "payment_gateway_bank": "Example Bank",  "payer_email": "user@example.com",  "payer_mobile": "1234567890",  "payer_card_brand": "Visa",  "payer_device": "device_id_123",  "payer_browser": "Chrome",  "payee_id": "payee_123"}
```

Response

```
{  "transaction_id": "txn_2443",  "is_fraud": false,  "fraud_source": "model",  "fraud_reason": "",  "fraud_score": 0.35329857887144267}
```

✔ No Fraud Detected

Transaction ID: txn_2443

Fraud Score: 35%

Detection Source: Model

Hybrid Model Results

<https://execute-4-0.vercel.app>

```
dhruv@MacBook-Pro-2 ai model % python3 app/model_training.py
[✓] Engineered features          9
[✓] Encoded: transactio_channel 1      e      t
[✓] Encoded: payer_email
[✓] Encoded: payee_ip_anonymous 6      8      6
[✓] Encoded: payer_mobile      0
[✓] Encoded: transaction_id
[✓] Encoded: payee_id          6      8
[✓] Train Distribution Before Resampling: {0: 110665, 1: 7}
[✓] Train Distribution After SMOTEENN: {1: 110656, 0: 110637}
[✓] Isolation Forest trained & saved

Training: RANDOM_FOREST          4
Training time: 80.77 secn_ds
[✓] Auto-tuned threshold on validation: 0.557
      precision    recall  f1-score   support

         0         1      0.25      0.50      0.33      27666
         1         0         1.00      1.00      1.00         2

 accuracy          0.62          0.75          0.67      27668
 macro avg          0.62          0.75          0.67      27668
weighted avg          1.00          1.00          1.00      27668

ROC AUC Score: 0.7475963276223523
Precision-Recall AUC: 0.06347592582437517
[✓] Saved random_forest_model.pkl
```

```
Training: LOGISTIC_REGRESSION          1
Training time: 0.51 seconds
[✓] Auto-tuned threshold on validation: 0.9
      precision    recall  f1-score   support

         0         1      0.00      1.00      0.00      27666
         1         0         1.00      0.96      0.98         2

 accuracy          0.50          0.98          0.49      27668
 macro avg          0.50          0.98          0.49      27668
weighted avg          1.00          0.96      0.98      27668

ROC AUC Score: 0.9670353502494036
Precision-Recall AUC: 0.001021600580815394
[✓] Saved logistic_regression_model.pkl

Training: NEURAL_NETWORK          5
Training time: 10.42 seconds
[✓] Auto-tuned threshold on validation: 0.671
      precision    recall  f1-score   support

         0         1      0.17      0.50      0.25      27666
         1         0         1.00      1.00      1.00         2

 accuracy          0.58          0.75          0.62      27668
 macro avg          0.58          0.75          0.62      27668
weighted avg          1.00          1.00          1.00      27668

ROC AUC Score: 0.9996024000578326
Precision-Recall AUC: 0.0881578947368421
```

```
Training: XGB00ST
Training time: 0.89 seconds
[✓] Auto-tuned threshold on validation: 0.345
      precision    recall  f1-score   support

         0         1      0.14      0.50      0.22      27666
         1         0         1.00      1.00      1.00         2

 accuracy          0.57          0.75          0.61      27668
 macro avg          0.57          0.75          0.61      27668
weighted avg          1.00          1.00          1.00      27668

ROC AUC Score: 0.9991505819417336
Precision-Recall AUC: 0.05329457364341085
[✓] Saved xgboost_model.pkl

Training: ENSEMBLE
Training time: 82.2 seconds
[✓] Auto-tuned threshold on validation: 0.443
      precision    recall  f1-score   support

         0         1      0.17      0.50      0.25      27666
         1         0         1.00      1.00      1.00         2

 accuracy          0.58          0.75          0.62      27668
 macro avg          0.58          0.75          0.62      27668
weighted avg          1.00          1.00          1.00      27668

ROC AUC Score: 0.9791621484855056
Precision-Recall AUC: 0.042319029780149085
[✓] Saved ensemble_model.pkl

Hybrid supervised + anomaly detection pipeline comple Spoti
```

Thank You So Much

TEAM NAME - SPARK
DELHI TECHNOLOGICAL UNIVERSITY

TEAM MEMBERS:

ARYAN GOEL

DHRUV MEERWAL

